Employee Raise Issue – Record Producer & Service Portal Integration Guide

Capstone Project Documentation

Team ID: 23KD1A0571 · 21-10-2026

1. Introduction

Project Title: Employee Raise Issue – Record Producer & Service Portal Integration Guide

2. Project Overview

Purpose: To give employees a guided, trackable way to raise an IT issue on the ServiceNow platform, from a Service Portal page to a validated, auto-numbered record.

Features: Scoped application, custom table with auto-number, UI Policies, dependent Category–Subcategory choices, a Record Producer form and a custom Service Portal with two widgets.

The Employee Raise Issue solution is a ServiceNow application that replaces informal issue reporting with a structured form. An employee opens the Requesting portal, selects the *Raise an Issue* widget and lands on the *Raise Employee Issue* form, where the requester, category, subcategory, state and short description are entered using clear choices.

Behind the form, a custom table named *Employee raises issue* stores every submission. Each record receives its own auto-generated number. UI Policies make the short description mandatory when the state is New and keep the number field read-only, and the subcategory options on the table form depend on the selected category.

The portal side is built with a dedicated Service Portal, a portal page and two custom widgets: a Commercial widget that describes the service and a Link redirect widget that opens the Record Producer. Everything is kept inside a scoped application called *Employee Center*. Overall, the project shows how standard ServiceNow features can be combined to give a standardized, accurate and easy-to-track issue submission experience.

3. Architecture

Frontend: ServiceNow Service Portal (“Requesting portal”), portal page and widgets built with HTML, CSS and an AngularJS client controller.
Backend: Record Producer with Map to Field, UI Policies and a dictionary dependency.
Database: ServiceNow table “Employee raises issue”, with references to sys_user and sys_user_group.

The solution is built in layers so that the user interface, the form rules and the data are kept apart.

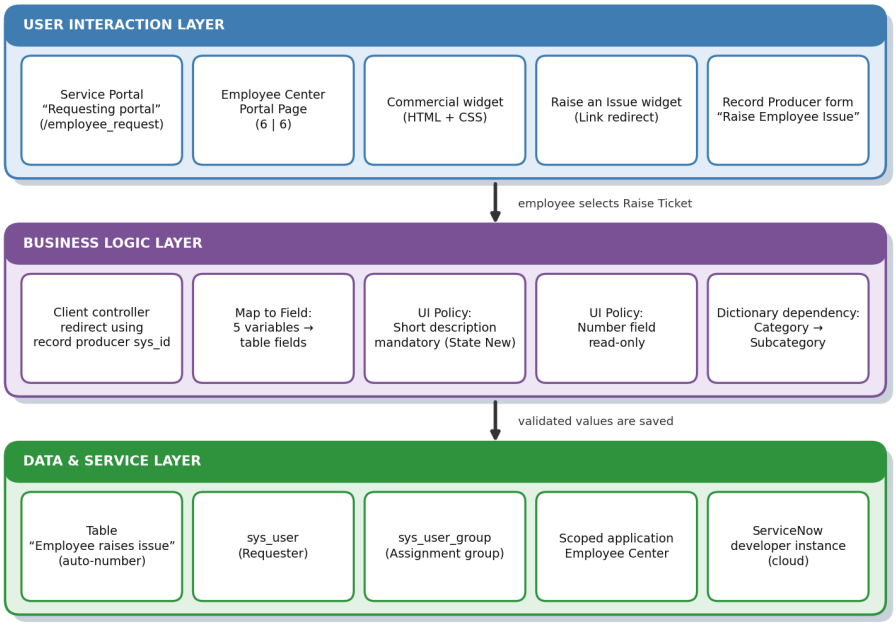


Figure 3.1 – Layered architecture of the solution

1. Presentation Layer (Frontend)

The Requesting portal uses the Employee Center homepage layout, the Employee Center Coral theme and the Employee Center menu. A 6 | 6 container holds the two widgets. The Link redirect widget uses a client controller to open the Record Producer, so the employee reaches the form with one click.

2. Service Catalog Layer

The Record Producer *Raise Employee Issue* is published in the Service Catalog under the Service category. Its five variables are mapped to the table fields and create the record when the form is submitted.

3. Form Rules Layer

Two UI Policies handle the field rules (Short description mandatory when State is New, Number read-only), and a dictionary dependency limits the Subcategory choices by Category.

4. Data Layer (Database)

The custom table *Employee raises issue* stores Number, Requester, Category, Subcategory, Short description, State, Priority and Assignment group. Requester refers to sys_user and Assignment group refers to sys_user_group.

4. Setup Instructions

Prerequisites: A ServiceNow developer instance with access to ServiceNow Studio, Service Catalog and Service Portal configuration.
Installation: Create the scoped application, the custom table, the UI Policies and dependency, the Record Producer, the portal, the page and the two widgets, in that order.

Step 1: Create the Application

- Navigate to All menus → App Engine → ServiceNow Studio.
- Click Create, then App.
- Name: Employee Center; Scope: Scoped; click Continue.
- Return to the home page and reload; the application appears as the current application.

Step 2: Create the Custom Table and Fields

- Navigate to System Definition → Tables and click New.
- Label: Employee raises issue (the name auto-populates).
- Under Controls, enable Auto number and save.
- From Related links choose Show Form, then Configure → Form Design and add the fields below.

Field label	Type	Notes
Number	Auto-Number	—
Requester	Reference	sys_user
Category	Choice	Network, Hardware, Software, Access
Subcategory	Choice	VPN, Laptop, Server, Forgot password
Short description	String	—
State	Choice	New, In progress, Resolved, Closed
Priority	Choice	Critical, High, Medium, Low
Assignment group	Reference	sys_user_group

Step 3: UI Policy – Short Description Mandatory

- Navigate to All menus → UI Policies and click New.
- Table: Employee raise issue; filter condition: State is New. Save.
- In UI Policy Actions add a new action: Field = Short description, Mandatory = True, Visible and Read only unchanged. Submit and update.

Step 4: UI Policy – Number Read-only

- Create another UI Policy on the same table with no filter condition.
- Add a UI Policy Action: Field = Number, Read only = True. Submit and update.
- A Dictionary override can be used as an alternative.

Step 5: Category → Subcategory Dependency

- Right-click Subcategory on the table form and choose Configure Dictionary, then open the Advanced view.
- Set Dependent field = Category.
- In the Choices related list, give each Subcategory choice the backend value of its Category: Network → VPN, Hardware → Laptop, Software → Server, Access → Forgot password.

Step 6: Create the Record Producer

- Navigate to Service Catalog → Catalog Definitions → Record Producers and click New.
- Name: Raise Employee Issue; Table: Employee raise issue; add a description.
- Accessibility: Catalog = Service Catalog, Category = Service; add a picture; save.
- In Variables, create five variables with Map to Field enabled and orders 100, 200, 300, 400, 500.

Order	Question	Type	Maps to
100	What is the Requester name	Reference (sys_user)	Requester
200	Which Category	Choice	Category
300	What is Subcategory	Choice	Subcategory
400	Which State is this issue	Choice	State
500	Short description	Single line text	Short description

Step 7: Create the Service Portal

Navigate to Service Portal → Service Portal Configuration and click New with these values:

Setting	Value
Title	Requesting portal
URL suffix	employee_request
Homepage / KB home page	ec_standard_home / esc_knowledge_home
Login page / 404 page	login / 404
Main menu / Theme	Employee Center Menu / Employee Center Coral Theme
Logo	Uploaded image

Step 8: Create the Portal Page

- Navigate to Service Portal → Pages and click New.
- Title: Employee Center Portal Page; ID: employee_center_portal_page; save.

Step 9: Create the Left Widget (Commercial widget)

- Open the Widget Editor and choose Create a new widget; name it Commercial widget.
- Add the HTML template and the CSS. The CSS gives the card a green–white–blue gradient, a floating heading (3 s loop), staggered slide-in list items and a rotating glow (8 s), with a lift on hover.
- Use Enable preview to check the result and save.

Step 10: Create the Right Widget (Link redirect)

- In the Widget Editor create a widget named Link redirect with HTML, CSS and a client script.
- The CSS uses a background photo with a blue-to-dark gradient overlay, a white pill button and a rotating glow (10 s).
- In the client script, paste the Record Producer sys_id into RECORD_PRODUCER_SYS_ID and save.

```
<a href="javascript:void(0)" class="raise-ticket-widget"
   ng-click="c.openRecordProducer()">
  <div class="overlay">
    <h2>Raise an Issue</h2>
    <p>Submit your request or complaint</p>
    <span class="premium-btn">Raise Ticket →</span>
  </div>
</a>

function ($scope, $location) {
  var c = this;
  var RECORD_PRODUCER_SYS_ID = 'ae2cd61183433210de6fc590ceaad3a4';

  c.openRecordProducer = function () {
    $location.url('sp?id=sc_cat_item&sys_id=' + RECORD_PRODUCER_SYS_ID);
  };
}
```

Step 11: Add the Widgets to the Page

- Open the Designer under Service Portal Configuration and select the Employee Center homepage.
- Add a container with a 6 | 6 split.
- Drag the Commercial widget to the left and the Link redirect widget to the right.

Step 12: Test the Solution

Open the portal URL, scroll to the widgets, select Raise Ticket, fill in the form, submit it and check the resulting record and its field restrictions (see Section 10).

5. Folder / Component Structure

The components of the project are organized inside the scoped application as follows.

```
Employee Center  (scoped application)
│
├── Data
│   ├── Employee raises issue  (custom table, auto-number)
│   │     Number · Requester · Category · Subcategory
│   │     Short description · State · Priority · Assignment group
│   ├── sys_user  (Requester)
│   └── sys_user_group  (Assignment group)
│
├── Form rules
│   ├── UI Policy: Short description mandatory when State is New
│   ├── UI Policy: Number field read-only
│   └── Dictionary dependency: Category → Subcategory
│
├── Service Catalog
│   └── Record Producer: Raise Employee Issue
│         5 variables (orders 100–500), Map to Field
│
└── Service Portal
    ├── Portal: Requesting portal  (suffix: employee_request)
    ├── Page: Employee Center Portal Page  (employee_center_portal_page)
    └── Widgets
        ├── Commercial widget   (HTML, CSS)
        └── Link redirect       (HTML, CSS, client script)
```

Key structural principle: UI layer → Service Portal and widgets; Catalog layer → Record Producer; Rules layer → UI Policies and dependency; Data layer → custom table.

6. Running the Application

Open the portal URL → select Raise Ticket → submit the form → the record is created with an auto-number.

Step 1: Access the Service Portal

The employee opens the portal URL (*https://<instance>.service-now.com/employee_request*). The home page shows My Recent Items, the search bar and, further down, the two widgets.

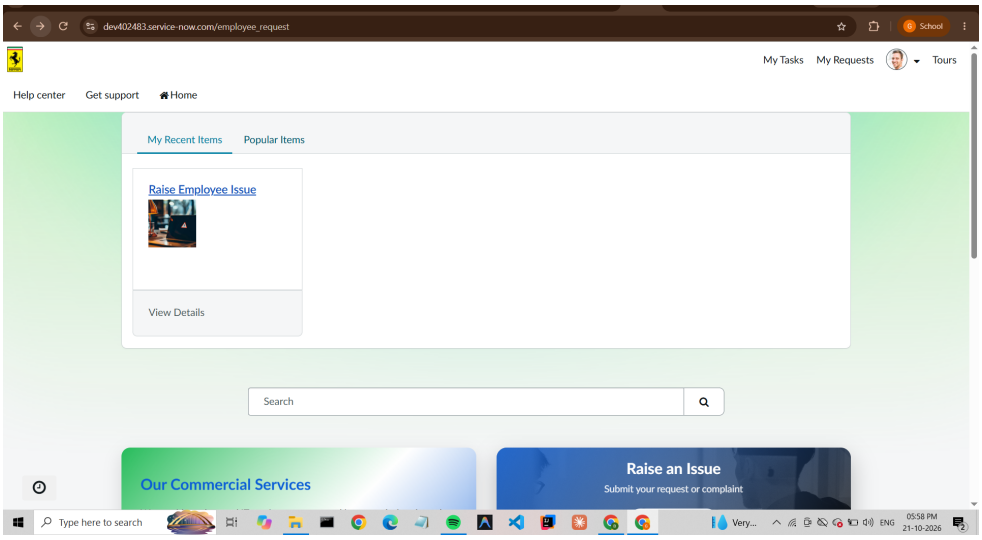


Figure 6.1 – Requesting portal home page

Step 2: Use the Raise an Issue Widget

Scrolling down shows the Commercial widget and the Raise an Issue widget above the Quick links and My active items. Selecting *Raise Ticket* → opens the Record Producer.

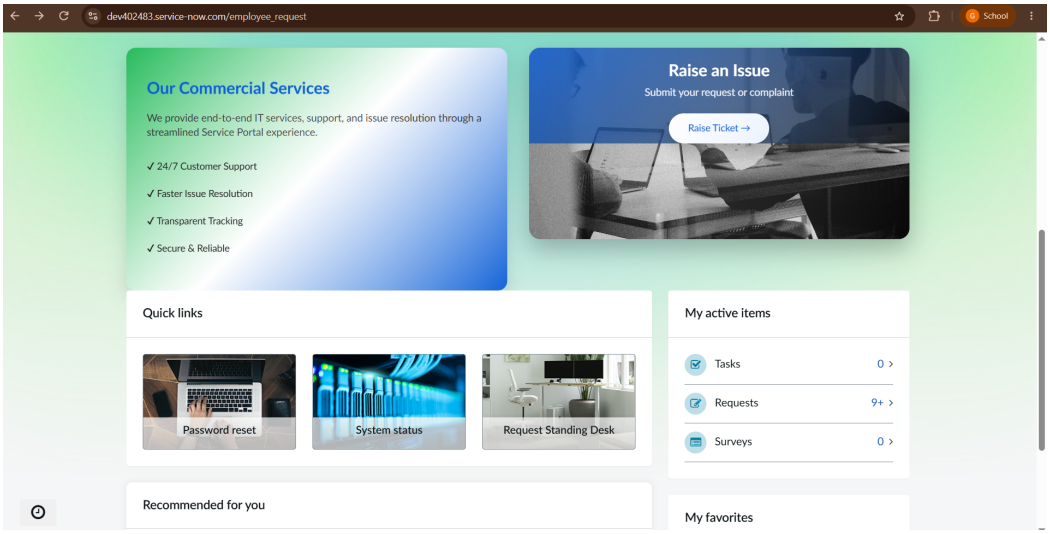


Figure 6.2 – Commercial widget and Raise an Issue widget

Step 3: Fill in the Form

The employee enters the requester, then chooses the category, subcategory, short description and state.

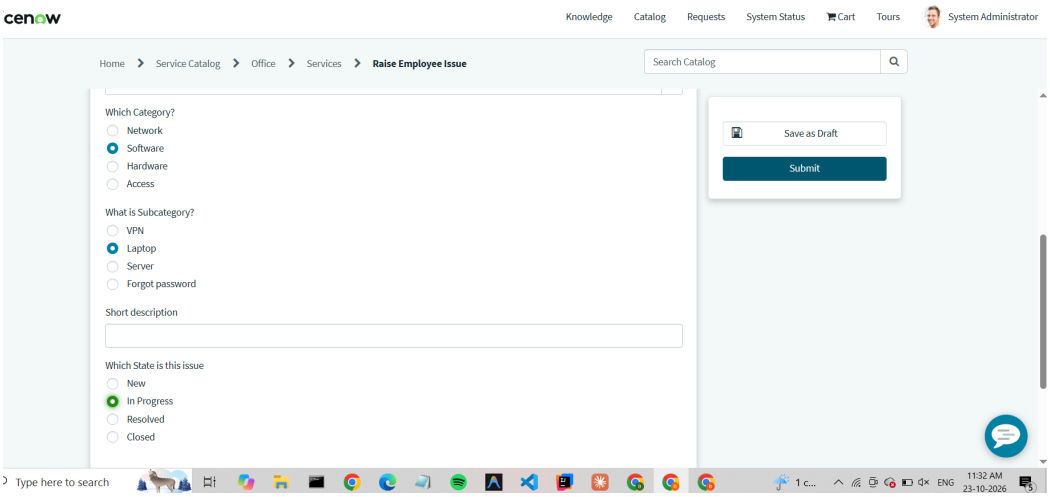


Figure 6.3 – Record Producer form with category, subcategory, short description and state

Step 4: Submit and Record Creation

After Submit, a record is created in *Employee raises issue* and receives its auto-generated number.

Step 5: Field Restrictions

On the record, the Number field is read-only, the Requester is marked as required, and Short description is mandatory whenever the State is New.

7. API / Automation Logic

This project does not call any external API. The automation uses features of the ServiceNow platform itself: the Record Producer and its Map to Field settings, UI Policies, the dictionary dependency and a small client controller in the Link redirect widget. The only external resource is the background image of the Raise an Issue widget, which is loaded from an image URL in the widget's CSS.

1. Internal Automation (ServiceNow Platform)

- The Record Producer turns the five variables into one record in the custom table.
- The table's auto number generates the issue number.
- UI Policies enforce the mandatory and read-only rules on the form.
- The dictionary dependency limits Subcategory options by Category.

2. Portal Redirect

- Clicking Raise Ticket calls openRecordProducer() in the client controller.
- The controller redirects with $location.url to sp?id=sc_cat_item&sys_id=<Record Producer sys_id>.

3. Data Flow Summary

Employee → Raise an Issue widget → client controller redirect → Record Producer form → submit → record in "Employee raises issue" → auto-number assigned.

8. Authentication

1. Authentication Mechanism

Users are authenticated by the ServiceNow platform. The portal is configured with the standard *login* page, so employees sign in with their ServiceNow account before using the portal. The Requester field on the form refers to the sys_user table. The screenshots in this document were taken while signed in as a System Administrator.

2. Access Control

The project steps do not define custom roles or ACLs, so access relies on the platform's default behaviour. Defining roles and access rules for the custom table is listed as a possible enhancement.

9. User Interface

The interface is built on the ServiceNow Service Portal with the Employee Center theme and two custom widgets.

1. Portal Home Page

The page shows the logo, the My Tasks / My Requests links, the Help center, Get support and Home links, My Recent Items with the Raise Employee Issue item, and a search bar.

2. Widgets

The Commercial widget presents "Our Commercial Services" with four points (24/7 Customer Support, Faster Issue Resolution, Transparent Tracking, Secure & Reliable). The Raise an Issue widget uses a photo background with a gradient overlay and a Raise Ticket button. Both lift and animate when the pointer moves over them.

3. Record Producer Form

The form shows the Raise Employee Issue title with its picture, the Requester lookup, radio buttons for Category, Subcategory and State, a Short description box, and the Submit and Save as Draft buttons.

4. Issue Record

The saved record opens in the “Employee raises issue” form with Number, Requester, Category, Subcategory, Short description, State, Priority and Assignment group.

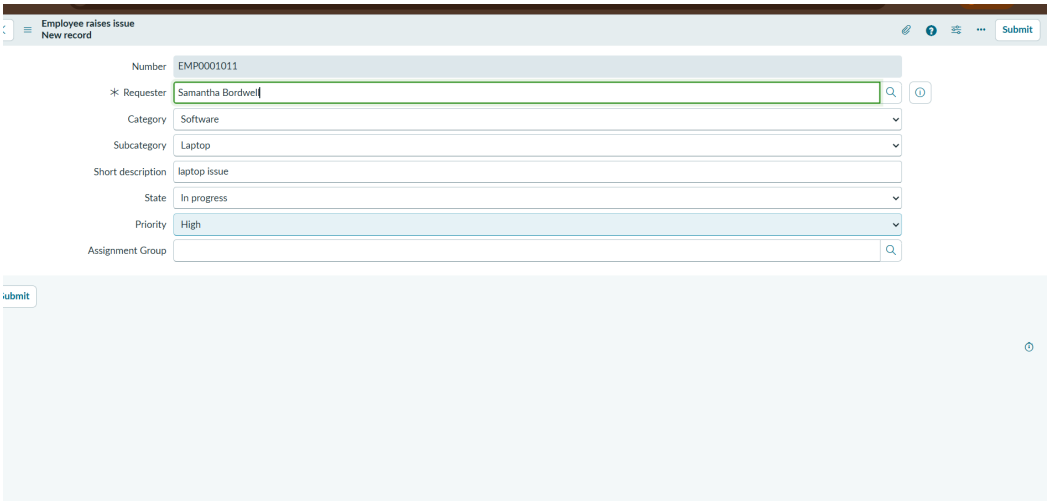


Figure 9.1 – “Employee raises issue” record form (Number is read-only)

5. User Experience Design

- **Simplicity:** one click from the widget to the form.
- **Consistency:** the portal reuses the Employee Center layout, theme and menu.
- **Clarity:** predefined choices instead of free text for category, subcategory and state.
- **Visual feedback:** animated widgets draw attention to the Raise an Issue action.

10. Testing

The solution was validated through manual testing of the full flow, from the portal to the saved record.

1. Test Scope

- Requesting portal home page
- Commercial and Raise an Issue widgets
- Record Producer form
- Record creation and auto-number
- UI Policies (mandatory and read-only fields)
- Category–Subcategory dependency

2. Test Scenarios and Cases

Test case	Input	Expected output	Result
1. Portal access	Open /employee_request	Portal home page loads with menu, recent items and search	Pass
2. Widgets display	Scroll down the portal page	Commercial widget and Raise an Issue widget are shown	Pass
3. Widget movement	Move the pointer over the widgets	Widgets float and animate	Pass
4. Open the form	Select Raise Ticket	Raise Employee Issue form opens	Pass
5. Form variables	View the form	Requester, Category, Subcategory, Short description and State are available	Pass
6. Record creation	Submit the form	Record is created with an auto-generated number	Pass
7. Read-only number	Open the record	Number field is read-only	Pass
8. Mandatory field	Set State to New	Short description becomes mandatory	Pass
9. Dependency	Select Network as Category on the table form	Only VPN is listed as Subcategory	Pass

3. Result Summary

All 9 test cases passed (9 of 9). No defects were recorded during testing.

11. Known Issues

The following limitations were noted for the current implementation.

1. Portal Form Does Not Filter Subcategory

The dependency is configured on the table form. On the Record Producer form the Category and Subcategory are separate variables that are not linked, so all four subcategories are shown whatever Category is chosen.

2. Priority and Assignment Group Are Not Collected on the Portal

The Record Producer has five variables. Priority and Assignment group exist on the table but are not asked for on the portal form.

3. Record Producer sys_id Is Stored in the Widget

The Link redirect widget keeps the Record Producer sys_id in its client script. If the Record Producer is recreated, the new sys_id must be pasted into the widget.

4. External Background Image

The background image of the Raise an Issue widget is loaded from an external URL. If the URL is unavailable, the image does not appear.

5. Performance and Access Rules Not Tested

Response times and behaviour under heavy use were not measured, and no custom roles or ACLs were defined in this project.

12. Future Enhancements

The solution can be extended further to address the limitations above.

1. Dependent Choices on the Portal Form

Linking the Category and Subcategory variables, for example with select boxes and a catalog client script, would make the portal form filter subcategories the same way as the table form.

2. More Fields on the Record Producer

Adding Priority and Assignment group as variables would let each issue arrive on the table fully described.

3. Roles and Access Rules

Defining roles and ACLs for the custom table would control who can create, view and manage issue records.